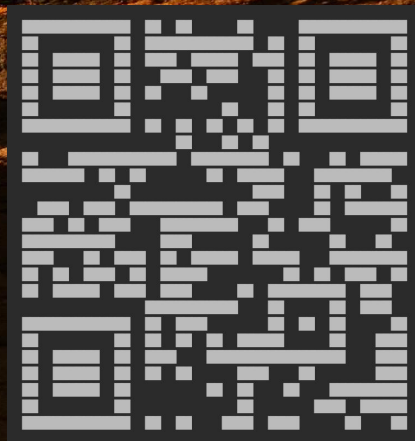


Low Latency Live Video Streaming

Prepared for syd<video>
ICC Sydney September 2020

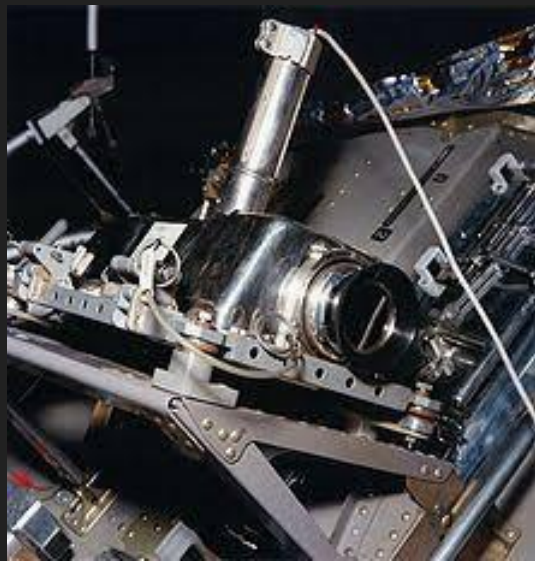
by Kevin Staunton-Lambert
@kevleyski

pyrmontbrewery.com



what we mean by live latency... July, **1969**

Delay between getting light into glass of the camera onto the glass of the viewers



3s time delay
(10fps@300)



#JusticeForHoneysuckle

what we mean by live latency... July, 2019

Delay between getting light into glass of the camera onto the glass of the viewers



**42s time
delay**
(25fps@1080)



#50YearsOfProgress :-)

why do we even care?

Social Media/eSports - shared emotion gets lost+kinda weird

Bizzare Hangouts/Adult interactive entertainment “VR issues”

Live Sports spoilers, hearing a ‘no try’ given that I’m still mentally helping the ref to decide upon - live score alerts

Questionable fairness in interactive betting/online auctions

News/Webinar - pretty awkward Q&A audience interactions

... and the problems get worse with 4K live streaming

who are the usual suspects...

Camera live output - raw vs compressed video (e.g. JVC 1.6s)

Outside broadcast uplinks - satellite hops vs optic fibre

Adaptive Bitrate transcodes - slow as your best rendition

Media file packagers, CDN file upload and propagation

Decoders (player apps, MSE SW vs HW, DRM CENC/EME)

External factors like SSAI Ad Injectors, live profanity buffers

how can we fix that?

Throw \$ at the problem...

- *Buy* better cameras - reduce compression/buffer time
- *Buy* faster hardware encoder GPU/FPGA + SSD storage
- *Buy* more live encoders (transcoders) + hw encryption
- *Pay* for better uplink/downlink/CDN bandwidth
- *Pay* to support the best solutions for *every* player type

how else can we fix things reducing live latency?

Some emerging technologies that help make things better...

- Improving codecs - better balance between hardware encoding/decoding - e.g. AV1 rav1e/dav1d, VVC
- More efficient media packaging, CMAF+DASH, LHLS
- Elastic cloud compute - scaling up/down to demand
- Better transport options, reusing connections, FEC
- Various proprietary solutions

what can you do to reduce your live latency...

Many options... also depends on the size of your audience

But first, a quick recap on the evolution and some common concepts and techniques which contribute to the emerging technologies that are used to reduce streaming media latency today...

TCP vs UDP - recap of the transport layer options

TCP (Transport Control) and UDP (User Datagram) are the two transport layer protocols that sit on the IP (Internet Protocol) packet network

TCP guarantees data gets to its final destination even if there are problems along the way, which is great for application layer protocols such as HTTP and TLS (SSL) but this comes with some major overheads like congestion/stall recovery/resend, handshake setup, round trip ack

UDP is more fire and forget - if there is an issue along the way bad luck

UDP `udp://` Near zero latency (1980's fire and pray)

When it works, it's an efficient way to transfer data over packet networks, NICs can be optimised for media broadcast via **multicast** IP, plus you get control over `fifo_size/pkt_size`
Typically you push to destination IP or route via multicast IP
But in reality UDP is particularly **error prone** - expect media glitches due to packet loss and packet reordering over net
UDP does not scale well over Internet distribution, at least not cheaply when compared with regular HTTP CDN solutions
... but no security and hey, you'll need an app for all this too

UDP **rtp://** **RTP** (Realtime Transport Protocol)

RTP is pretty common, Apple FaceTime for example utilises this and other tech like WebRTC wrappers it to some extent

It comes with benefits of UDP for reduced overheads and multicast - and adds some smarts to detect packet loss and so automates re-sending and correcting the ordering of packets to avoid jitter in the audio/video bitstream

RTSP adds session control to RTP, this is where SDP originates

TCP rtmp:// RTMP Real Time Messaging Protocol

A great solution where latency is largely coming from the camera encoder to **push** compressed video to a distribution

But RTMP is (almost always) P2P Unicast TCP/IP, so...

Need more relay ports? get more/bigger servers!

Want to load balance/bypass firewalls? Maybe RTMPT (HTTP)

Ok for eSports/live betting with small actual paying punters
meanwhile those just watching along in higher latency (if
legally permitted) ... scaling up to Grand Final Footy? Nope!

... also, **Adobe Flash/FLV** player in your app ain't gonna fly

TCP http:// Lots of segmented media files on a CDN

Rather than streaming muxed (combined audio/video/captions) media in a continual torrent of data, media can also be chopped up into byte sized “segments” and distributed as separate files over the internet

Big advantages to this are traditional HTTP based content delivery networks (CDNs) can quickly propagate these objects worldwide very efficiently and ABR become easier to juggle
But which files? where to get them, what if there is a problem

TCP wss:// WebSockets raw secure media pipes

Can achieve really excellent Ultra low latency (<200ms),
sending raw live video into a WebSocket *directly* into web
browser/MSE - TCP covers errors, but can (and does) buffer
Renders video using hardware (MSE) via <video> tag
But also HTML5 <canvas> which means WebGL 3D surfaces
and cool GLSL fragment shader effects/overlays etc
Also easy to integrate, WebSockets are well supported in all
common programming languages too, apps like TikTok etc

UDP srt:// SRT (Secure Reliable Transport)

UDP with end-to-end (AES/DVB) encryption + a big bonus...

Includes forward error correction (FEC) which is utilised in traditional digital broadcast. Some additional data is added so the receiver can auto patch up common errors (NAK packets). Algorithm also automatically retries to refetch segments again if time frame allows incase FEC above fails

... open source protocol but no takers into the MSE

(more an FFmpeg/Gstreamer/VLC) RedBee claim **3.5s**

(kudos to Haivision and Wowza for open source)*

UDP rist:// RIST (Reliable Internet Stream Transport)

RTP w/ SMPTE-2022 low latency solution from the Video Services Forum (broadcasters / independent vendors)
Encoder (sender) and Decoder (receiver) send packets via relays message server, which resends on not acknowledged packets (NACT/RTCP) based on number of retries set by the decoder - this effectively self recovers the bitstream from typical UDP packet loss and ordering sequence issues
Added security via DTLS (i.e. certificate-based authentication)

WebRTC (Web Real-Time Communication)

W3C (standard) WebRTC, similar in nature to WebSockets so ultra low latency, but once established is peer-to-peer direct To handle external delivery via Internet it needs proxies to work (ICE/TURN/STUN) to establish/maintain connectivity Depending on connection can be UDP or TCP, fun stuff like NAT traversal and SDP session management implies some operational headaches and so expect ongoing support costs

HLS Legacy, how things were (HTTP Live Streaming)

Adaptive Segment file based delivery via TCP/HTTP1.1

- MPEG2 Transport Stream (TS) files - often muxed sometimes separated audio e.g. separate AAC files
- Recommended segment length 10s (most used 6s)
- Players request 3 (possibly AES encrypted) segments and decode them before playback can begin
- Overall latency typically more than **30 secs** ... so not great

HLS Legacy (limitations around reducing latency)

Shorter segments files *do* help but with side effects...

More files uploaded on the CDN = additional charges

Greater overheads as less compression + TCP connections

Less data + transport delays leads to more buffering which means longer playlists which undoes some of your good work
my own tests give lowest watchable latency around **9s**

Apple's player seems to be most robust to errors but is closed source - hls.js ok but is MSE browser only, ExoPlayer not great, others also fail often

MPEG-DASH (Dynamic Adaptive Streaming over HTTP)

Provides better flexibility over HLS where live streams are now organised into AdaptationSets, segment templates and separates timing info away from the media, add in presentation availability windows and can handle multi-DRM and importantly it's standards based (HLS remains an RFC) mediaRanges supported since conception allowing more efficient byte-range transfer over HTTP, whilst HLS more recently added this feature, it's a clumsier after thought

MPEG-DASH (Dynamic Adaptive Streaming over HTTP)

Other benefits over HLS is supporting multiple linear live events with HLS discontinuities is a royal pain in the bum, DASH to the rescue with multi-periods (and just maybe dynamic x-links will *actually* be supported in all players some day)

Also a large difference to early HLS is use of fMP4/ISOBMFF where MPEG-TS has no native MSE support in the browser* (* *actually DASH supports TS but seldom implemented outside of HbbTV/DVB-DASH*)

ISO BMFF Base Media File Format (MP4)

Standardised what is commonly known as MP4 container files

All data put in well defined 'boxes' (aka QuickTime 'atoms')

- Movie Box (moov) and Movie Fragment Box (moof)
- Media Data Boxes (mdat)
- ftyp, styp etc...

Separation of media initialisation vs segments removes *much duplication* of metadata about the media - e.g. *moov* box contains codec info - every TS segments duplicates such data

fMP4 Fragmented ISOBMFF

Large ISOBMFF files can be further partitioned into fragment files which are then easy - and so quicker/more efficient to

- generate (add new boxes whilst old ones being read)
- manipulate (easy to filter, skip over and go back to)
- serve (easy to pass boxes around via byte range requests)

Supported by *both* HLS *and* DASH means less bandwidth needed ⇒ faster upload times ⇒ lower CDN and storage costs and a slightly better overall bitstream compression

h2 HTTP/2 Sending less, sharing with PUSH (concat)

HTTP/1.1 limits 6 open connections, so juggling often needed

h2 removes this limit and also brings in HPACK which compresses the headers by indexing them (huffman) which reduces duplicates, combined with gzip of manifests reduces over bandwidth with streaming media

Dependency and waiting

When sending files, piggybacking by pushing other files sharing the same TCP/HTTP connection reduces overhead

codec what's the best codec for lowest latency?

Largely this down to what hardware your capture device might handle but reduced bitrates and GOP size is factor

Cameras often compress mjpeg as well as raw, both need encoding for distribution, but if camera can push H.264 then that's likely to yield the best glass-to-glass latency

Needs a balance for encode and decode, for example AV1 decoding (e.g. Snapdragon) in devices is becoming practical, but the encoder (e.g. SVT) remains heavy weight to keep up

newer low-latency streaming technologies

Some emerging technologies

- Better delivery - ACTE, HTTP/3 (QUIC), WebTransport

- LL-HLS (Apple Low-Latency HLS)

- ULL-CMAF (Ultra Low Latency CMAF)

- Codec improvements (more efficient bitstreams)

- Peer-to-Peer and Broadcast WebRTC, WHIP and WHEP

- Some other proprietary solutions, such as HESP and Psy

ACTE ABR for Chunked Transfer Encoding

Improves latency by reducing stalls by the player due to over or under buffering

Uses a sliding window approach to more accurately measure the available bandwidth the player can buffer efficiently

Calculated from response vs request time of the last three most contiguous chunks and this feeds into an old school recursive least squares (RLS) algorithm to predict the bandwidth into the future which pokes the ABR switcher

Implemented in dash.js today

TCP BBR Bottleneck Bandwidth RTT (congestion control)

TCP flow control, what BBR does is frequently probe the bandwidth and round trips and then paces the packets accordingly vs the regular Linux TCP stack mode (cubic) that reacts to things already going wrong (packets getting lost and so needing to be resent) so this better ensures our video packets flow better whilst increasing the likeliness they'll get there first time, thus improving latency

```
net.core.default_qdisc=fq
```

```
net.ipv4.tcp_congestion_control=bbr
```


UDP QUIC (Quick UDP Internet Connections)

Google's UDP based delivery (e.g. YouTube Live)
Utilises SPDY (now HTTP/2) multiplexed connections which allows sharing a pipeline avoiding overheads with opening/reopening sockets and so greatly reduces the TCP handshake latency and head of line blocking problems (less congestion)

Whilst QUIC stand alone was not widely supported outside of Google its part of HTTP/3 moving forwards and you'll find hidden in the settings that iOS14 has support for that today

h3 HTTP/3

HTTP/3 will include *best-of-breed* of parts from QUIC, SRT, RIST and WebRTC/ObjectRTC

Combined with CMAF packaging using agreed common standards makes it viable for others to then adopt moving forwards

... but then standards always take time to establish

WebTransport W3C API (bidirectional low latency)

WebTransport is HTTP/3 + encrypted and congestion-controlled best efforts communication API
Purpose in part is to fix issues where packet loss and order might not necessarily matter so much (head of line issue), allow things to move on smoothly rather than try and resend, FEC and so on
Uses QUIC to keep connections alive over UDP

WebRTC Peer2Peer and Broadcast ObjectRTC

There exist some interesting scaling opportunities to be had via peer sharing where the origin shares content into a browser and it becomes the new origin - others piggyback on that player instance and in turn become origin nodes.

Broadcast WebRTC is coming but my guess is never likely to be 'super bowl' event ready either, presumably the costs saved in scalable CDN distribution vs support costs to relay my own tests show WebRTC is prone to errors (jitter)

WHIP WebRTC-HTTP ingestion protocol

A simple HTTP POST based protocol that will allow WebRTC endpoints to ingest content into streaming services and/or CDNs and facilitate deployment

Perform a single shot SDP offer/answer so an ICE/DTLS session can be established between the encoder/media producer and the broadcasting ingestion endpoint.

Once the ICE/DTLS session is set up, the media will flow unidirectionally (no tracks or streams can be added)

WHEP WebRTC-HTTP egress protocol

Similar in nature to WHIP but for integration into non or partially supported WebRTC API devices as it's not always practical to support/document, e.g support player only

WebRTC ask bring other tech such as FlexFEC and RED which whilst could increase latency (redundancy overheads) it ensures less errors and so less reconnects and jitter

Salsify Codec/network integration

Shares video codec metrics with the network transport protocol allowing it to respond quickly to network condition to avoid provoking packet drops and queueing delays, network struggling? Hey calm your farm a bit encoders Optimizes the compressed length and transmission time of each frame, based on a current estimate of the network's capacity vs frame rate or bit rate. ... but ultimately needs a codec or a way into the MSE. (Stanford uni)

CMAF Common Media Application Format

Standard from *Apple*, *Microsoft* and *Akamai* that enforces fMP4 and encoding profiles used across **HLS** and **DASH**

Media structure becomes consistent, video must fit within a smaller profiles/level set of AV1/ HEVC (H.265)/AVC (H.264)
Audio is *never* multiplexed content (always separate files) and must be AAC-LC/HE-AAC at given rates

Subtitles must be TTML/WebVTT

Encryption (whilst optional) must be AES ctr ... and so on

ULL-CMAF Low Latency Chunked HTTP transfer

Here we also **chunk** the fMP4 fragments further by adding metadata about media byte ranges into the MPD or M3U8

Byte range info allows players to take smaller playable bites at the segment file still being generated rather than waiting then chewing on the whole segment before starting playback

Chunked HTTP transfer reduces overheads whilst allowing the encoder to append new 'boxes' to same segment file

HESP High Efficiency Streaming Protocol

Proprietary protocol enabling streaming services to be delivered at scale with a significantly reduced bandwidth and with a sub-second latency and improved stream startup (zapping) time

One of the mechanisms is actually my own idea - made public at Demuxed 2019 - around separating a I-frames track from the others, this would allow better CDN caching performance of a main stream (less I-frames) and perceived faster stream switching by grabbing nearest I-frame and have the decoder catch up to that using the main stream

LCEVC Low Complexity Enhancement Video Coding

Reduced latency through lower bitrate + faster start-up piggy backs on existing codec which can be your hardware encoder and decoders you might already target as its baseline encode
Calculates a delta between full resolution and the best your existing hardware encoder and decoders can muster
Sends delta as "enhancement" stream sideband to existing
Older decoders can still decode by ignoring the enhancement
Volumetric data (e.g. AR/VR point cloud) also in development
(browser DRM challenges, wasm decoder staggers startup)

audio Low latency audio codecs (live speech)

Audio being low bitrate is typically not a major contributor to latency per se, audio is easily chunked too but where more channels are being added for hi fi ambisonic (many channels for immerse use cases from multiple sources) this adds up. Some more recent codecs use machine learning to predict speech patterns to allow extremely low bitrates (< 3kbps). Lyra (Google) and Speex used in Meet and Duo. Satin (Microsoft) used in Teams and so likely Meta workplace.

Apple LLHLS Low-Latency HLS

Apple announced Low-Latency HLS ~~last month~~ at WWDC19

Version 9 of HLS brings in some new tags but remains **backwards compatible** for all earlier players - in that the new tags will be ignored (as per specification)

It works by adding partial media segment files into the mix, this can be CMAF fMP4, but Apple continues to support TS files in the form of partial MPEG2 TS transport segments too

Apple LLHLS Partial Segment Tags #EXT-X-PART

Partial TS segments are described in media playlist like this

```
# INDEPENDENT=YES tells the player  
# part 0 of segment 787 has an IDR  
# (starts with an independent I-Frame)  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=YES,URI="filePart787.0.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.1.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.2.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.3.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.4.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.5.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.6.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.7.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.8.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.9.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.10.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.11.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.12.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.13.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.14.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=YES,URI="filePart787.15.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.16.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.17.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.18.ts"  
#EXT-X-PART:DURATION=0.20000,INDEPENDENT=NO,URI="filePart787.19.ts"  
#EXT-X-PRELOAD-HINT:TYPE=PART,URI="filePart787.20.mp4"  
#EXTINF:3.96667,  
fileSequence787.ts
```

part 15 of segment **787** also starts with an independent I-Frame

Note, remains backwards compatible by also always including the entire segment **787** in the usual HLS way

Apple LLHLS Partial Segment TS Files

There's nothing particularly special about a partial TS, literally split on TS 188 packet syncbyte and can be split outside GOP

file**Sequence**787.ts *(is same as)*

```
cat filePart787.0.ts
```

```
filePart787.1.ts
```

...

```
filePart787.19.ts > fileSequence787.ts
```

First part always 0 of arbitrary 20 parts, requesting part 21 = part 0 of next seg (according to spec, Apple's LLHLS demo tools are buggy ;-)

Apple LLHLS HTTP/2 PUSH ?_HLS_push=1

HTTP/2 is now a requirement to push the segments file along with the playlist as when it becomes ready

Piggybacking the segments this way significantly reduces the overhead of establishing repeated TLS / TCP sessions

Playlists are also always compressed under HTTP/2 - streams with a long DVR window (large review scrub back buffer) this compression also reduces latency to download it

Noticable difference between HTTP 1.1 (as demoed to Mark)

Apple LLHLS Preload #EXT-X-PRELOAD-HINT

#EXT-X-PRELOAD-HINT

Lets the server tell the player client the upcoming (parital) segment that is not yet actually available

This avoids the requirement for HTTP/2 push/blocking mechanism having to wait for the segment to get ready where the client can request it until it's actually available

Apple LLHLS TLS 1.3

Transport Layer Security 1.3 is also a requirement, ultimately it has less handshake overheads

TLS false start (tolerates receiving TLS records on the transport connection early, before the protocol has reached the state to process them)

Zero Round Trip Time (0-RTT - resumed connection when certificate has been used before)

Apple LLHLS #EXT-X-SERVER-CONTROL

```
#EXT-X-SERVER-CONTROL:CAN-BLOCK-RELOAD=YES,CAN-SKIP-UNTIL=24,PART-HOLD-BACK=0.610
```

This tells the player that the server has the following capabilities...

CAN-BLOCK-RELOAD=YES: Mandatory, simply means I have **?_HLS...** support

CAN-SKIP-UNTIL=<seconds> I'll give you 24 seconds back on **?_HLS_skip=YES**

PART-HOLD-BACK=<seconds>: Indicates the recommended **live edge** time when playing. This must be at least 3 x PART-TARGET - we have 20 parts per 4 second segment, so 0.2 seconds per parts, so *hold player back for* $(3 * 0.2) < 0.61$ seconds

Apple LLHLS Delta Updates ?_HLS_skip=YES

#EXT-X-SERVER-CONTROL:CAN-BLOCK-RELOAD=YES,CAN-SKIP-UNTIL=24,PART-HOLD-BACK=0.610

Playlist optimised by only sending what changed in a given time window - here the server tells the player I'll give you the next 24 seconds from when you next call ?_HLS_skip=YES

Typically delta changes fit in single MTU making it more efficient to load the playlists

Large DVR windows (review buffer) become highly compressed and much faster to parse thus reducing latency

Apple LLHLS Blocking Playlist Reload ?_HLS_msn=

When requesting live media playlist, wait until the first segment is also ready and give me back both at same time (saving additional unnecessary HTTPS/TCP round trips)

```
GET https://lab.streamshark.io:10433/2M/part.php?_HLS_msn=23058
```

...blocking/waiting until filePart23058.x + fileSequence23058 becomes available...

```
#EXT-X-PART:DURATION=0.20000,URI="filePart23058.0.ts"
```

```
#EXT-X-PART:DURATION=0.20000,URI="filePart23058.19.ts"
```

```
#EXTINF:3.96667,
```

```
fileSequence23058.ts
```

Apple LLHLS Rendition Reports ?_HLS_report

#EXT-X-RENDITION-REPORT

Adds metadata to other media renditions to make switching between ABR faster

_HLS_report=<path> points to the Media Playlist of the specified rendition. (either relative to the URI of the Media Playlist request being made, or an absolute path on the same server. Multiple report parameters are allowed for different paths.

Apple LLHLS tools

mediastreamsegmenter *(updated)*

“tool listens for its input stream on a local port and packages it for HLS. It writes a single live Media Playlist with its corresponding Media Segments (including Partial Segments in Low-Latency mode). It can also perform segment encryption. It writes its output to the local filesystem or to a WebDAV endpoint.”

tsrecompressor (optional encoder)

“produces and encodes a continuous stream of audio and video, either programmatically (a bip-bop image derived from the system clock) or by capturing video from the system camera and microphone. It encodes the stream at several different bit rates and multicasts them as MPEG-2 Transport Streams to local UDP ports.”

golang and php playlist file generators

Apple LLHLS demo

Example Apple low latency live streaming
(requires tvOS13, iOS13 Beta 2/3 Safari or newer)

StreamShark Origin:

<https://lab.streamshark.io:10433>

Local Clock <https://lab.streamshark.io:10433/time.html>

Disclaimer: Roger Pantos' demo of this with Sydney went pretty awry
This is all brand new stuff, when writing these slides low-latency HLS wasn't even
available in iOS & iPadOS 13 beta 1, and no Safari doesn't have it either even today

Apple LLHLS demo - Fastly CDN **3.15s**

Seconds tick over exactly on the middle 3 appearing so $30.148 - 27.0 = 3.15s$



Apple LLHLS demo - explained

Playlist Delta Request

Part1678.18.ts

/lhls/2M/lowLatencyHLS.php?_HLS_skip=YES

/lhls/2M/filePart1678.19.ts

Last partial segment of 1678

/lhls/2M/filePart1678.20.ts

/lhls/2M/filePart1679.1.ts

First partial segment of 1679

/lhls/2M/filePart1679.2.ts

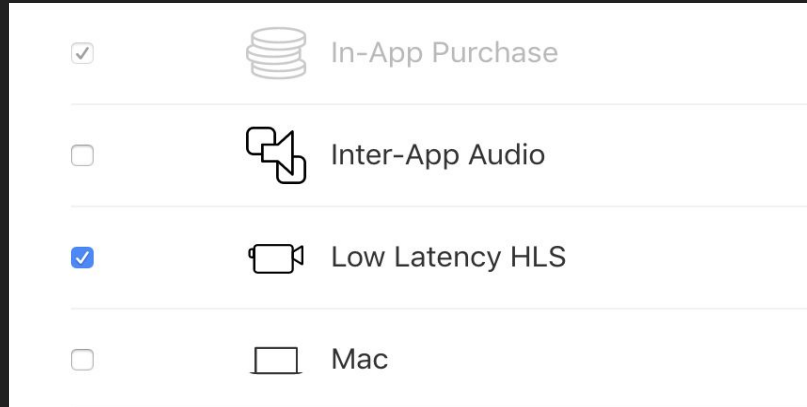
/lhls/2M/filePart1679.2.ts

Apple LLHLS Low Latency AVPlayer apps edits

Certificates, Identifiers & Profiles
new Low Latency HLS capability
(not enterprise yet)

(in Xcode need to add)

```
<plist version="1.0">
  <dict>
    <key>com.apple.developer.coremedia.hls.low-latency</key><true/>
```



AVPlayerItem now has some new properties to request how far from **live edge** we *ought* to be stay in vs what we *want* to try and stay within - too small and buffering is more likely for example - too long and you are not lowest latency

Community LHLS (a nod to the original LHLS)

Community lead initiative (Periscope, JW, Twitch and others)
Apple's 'not invented here' and their draconian AppStore requirements probably will more or less kill this as a standard
But avoids the CDN cache busting complication where Apple reserves ?_HLS query string - also CMAF/MSE better approach
Quick demo (Akamai/JW)

Just maybe a best-of-breed solution may result from all this
... but my guess is that's some pretty wishful thinking

WebAssembly (on the CDN edge)

CDNs can help reduce latency too when streaming large scale Apple Low-Latency manifests and partial TS segments can be generated at the local point-of-presence instead of origin

Fastly is an example of a CDN supporting compute on the edge where WebAssembly can be used to process manifests and potentially process input TS streams a chunk at a time - generating virtual files on-the-fly without uploading them

Use cases around SSAI to smooth out PTS/DTS/PCR timecode

IPFS (InterPlanetary File System)

IPFS is peer-to-peer and works by converting files into blocks
The content is SHA hashed which becomes its URI referenced
across the *universe* via IPNS using publish/subscribe methods

For media stream this works reasonably well because IPFS
gateway supports byte-range requests easy to repackage
Possible movie distribution - encode once, *share* to many
Checkout GT Systems doing good things in this area

streamline End-to-end reference/example

A reference system for end to end live streaming video.
Capture, encode, package, uplink, origin, CDN, and player

Works with SBCs like Raspberry Pi

= Affordable live encoders

+ Interesting security use cases (disposable encoders)

Next gen Raspberry Pi is expected to have M.2
(which means SDI input from DeckLink cards for example)

streamline building a \$35 live video encoder LIVE

Unboxing Raspberry Pi during
syd<video> Demuxed 24hr
meetup



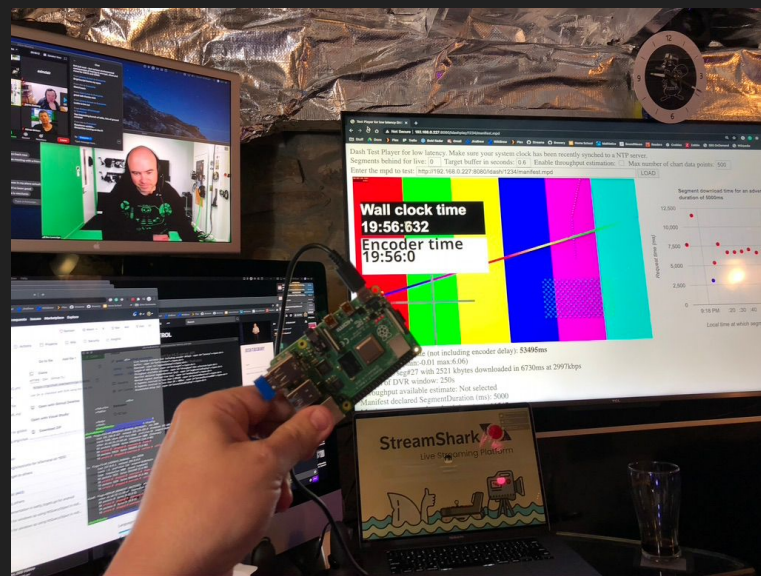
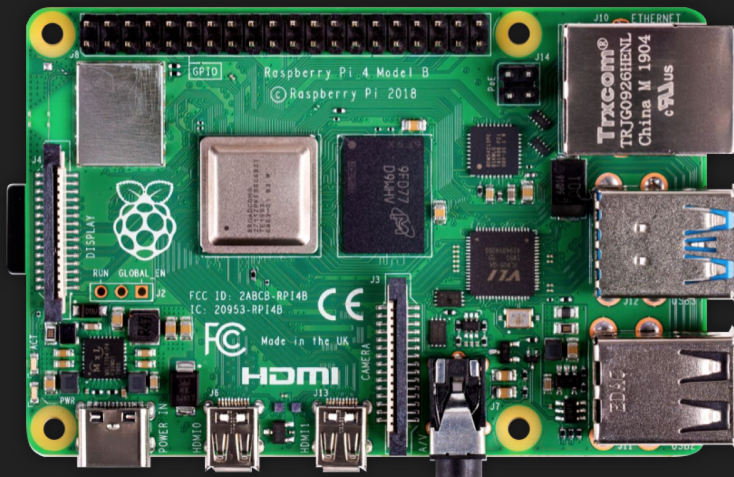
streamline building a \$35 live video encoder LIVE

git clone

<https://github.com/streamlinevideo/low-latency-preview>

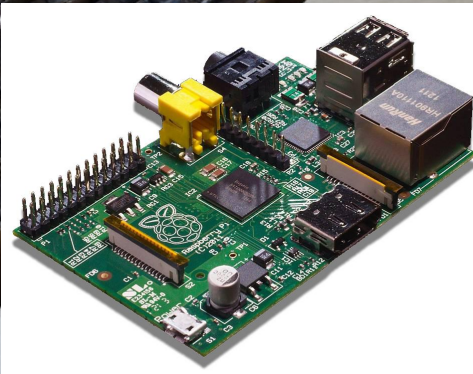
streamline rebuilt for ARM64

Up and running by mel<video>



streamline will it works on the original Pi?

Erm, maybe at really low resolution VideoCoreIV GPU approx original Xbox - so yes feasible but not very practical (out of time tonight, cross compiling for hard float ABI non trivial yada yada)



streamline building a \$35 live video encoder LIVE

zzzz's



streamline building a \$35 live video encoder LIVE

<http://pyrmont.live:1234/>

8:30am today

Adding \$12 HDMI to USB



streamline building a \$47 live video encoder LIVE

Problems and anecdote...

At night testing with outside cameras not much happens ;-)

With variable bitrate what worked well when it was dark in the morning when the sun comes up and pixels start to move around (bitrate spike)

Page swapping on SD card...

Pi slows down every now and then with serial I/O - get the 8G

streamline latency measurements

Typical 2.3s (1080 / 30fps)

Best case which seemed reasonably stable 800ms

Unstable but maybe ok at low bitrate/resolution 300ms

other excellent low latency solutions out there

Great results, often MPEG-TS over UDP solutions, so easy to manipulate/multicast but often there are other things to check... buyer beware

- Might be unencrypted only (probably means bitstream is manipulated)
- May require proprietary CDN to operate at speed
- Might need ports opening for the client
- May require their player to be integrated (so ongoing support, and Apple likes to screw with us from time to time by pushing back on this)
- Sometimes camera encoding is also a major factor in performance
- Closed sources typically hampers debugging production issues

players supporting low latency today (mid July 2019)

Chromecast MPL and CAF (Shaka) - no partial segments yet

HbbTV no - recent moves to HTML5/MSE this will improve

hls.js - community LHLS, slightly stunned by Apple right now

DASH.IF - CMAF supported, low latency mode option + ACTE

AVPlayer - iOS13 beta 2, AppleTV tvOS13 beta, **Safari soon**

Android ExoPlayer media range yes, LHLS in progress

Roku/TelstraTV - partials not supported, slow to keep up

Bitmovin, THEO - nothing public yet, closed source (JW actively support hls.js)

iOS14 also has h3 support (head to developer options)

Conclusions HLS or DASH, which is best?

Winamp playlists are easy to parse and manipulate vs complex XML/MPD files, well early HLS was easy anyway HLS has some major shortfalls around dynamic manifests - segment templates is desperately missing. Supporting SSAI is trivial with multi-period DASH vs HLS making dissimilar streams play leads to significant **encoder energy waste :-)** DASH has too many permutations to attempt to test everything, whereas HLS is comparatively straight forward DASH is open forum, which is good and bad, HLS continues to give Apple the upper hand always, historically not awesome

Conclusions The future - bridging the two beasts

With CMAF pushing to quantize the permutations around what should be allowed to climb onto an encoding ladder it opens the possibility of taking either HLS or DASH as an input stream and outputting the other on-the-fly

An edge use case for this is supporting MPEG-DASH on iOS

This would reduce packaging complexity and delivery overheads for the content providers, expect some debugging

Conclusions

In a nutshell things also revolve around TCP vs UDP and codec efficiency around how errors are tolerated (and so hidden)

TCP imposes overheads around setting up and maintaining said connections - but with bonus it's generally error free
UDP is crazy error prone, so basically the race is on find the most acceptable workaround for a/v bitstreams

Conclusions

Partial media segments over HTTP/3 seems to be the accepted trend around packaging and transporting the bitstream, the common themes being...

- Cross-platform, all player set-tops, SmartTVs, all browsers
- Scalable, runs over regular HTTP/CDN technology
- Supports CENC/EME (AES encrypted bitstreams)
- Open Source (no patents/royalties)

Conclusions

Good news is Apple, Microsoft and Google all seem to be in agreement e.g. MSE support finally arrived in iPadOS 13

Basically there is an obvious cost/benefit trade off

Lowering your latency ⇒ more expensive infrastructure

Monetising (live ad insertion) ⇒ more expensive encoders

... and, how bothered is the end viewer really anyway? If they were *that* bothered they'd have just gone to the game!? :-)

Thanks!

Skipped over something? sides are here

<https://tinyurl.com/>

yvr2rz8m

More on Raspberry Pi

<http://bit.ly/39tZcMm>

Also AV1 <https://goo.gl/pGnNgJ>

And WebAssembly <https://goo.gl/2ahsEY>

kev@pyrmontbrewery.com.au

